

Detección de Audio Falso (Deepfake) mediante un Modelo Híbrido CNN–BiLSTM

Diego Alfaro
a01703337@tec.mx

Mayo 2026

Resumen

Este documento describe la implementación de un modelo de aprendizaje profundo para la detección de audio falso (*deepfake audio*). El modelo combina una red neuronal convolucional (CNN) con una red recurrente bidireccional de memoria a largo y corto plazo (BiLSTM), siguiendo la arquitectura híbrida propuesta por Chapagain et al. [1]. Se documentan cuatro aspectos del cuaderno `Fake_Audio_Detecion_Model.ipynb`: (1) el preprocesamiento del conjunto de datos, (2) la división de los datos en entrenamiento, validación y prueba, (3) la implementación del modelo, y (4) las evaluaciones y métricas empleadas.

1. Introducción

La detección de audio falso busca distinguir entre grabaciones de voz auténticas (*real*) y grabaciones sintetizadas o manipuladas artificialmente (*fake*). El enfoque adoptado en este trabajo transforma cada señal de audio en una representación tiempo–frecuencia (espectrograma de Mel) y la procesa con un modelo híbrido CNN–BiLSTM. Las capas convolucionales extraen patrones locales del espectrograma, mientras que las capas recurrentes bidireccionales modelan la dependencia temporal de la señal. Esta arquitectura está inspirada en el modelo híbrido CNN–BiLSTM con mecanismo de atención de Chapagain et al. [1] y en redes en cascada para deepfakes de audio como ABC-CapsNet de Wani et al. [2].

2. Preprocesamiento del conjunto de datos

El conjunto de datos utilizado es *The Fake-or-Real (FoR) Dataset*, que contiene grabaciones etiquetadas como reales o falsas. Cada archivo de audio se convierte en un espectrograma de Mel en escala de decibeles antes de alimentar al modelo. Los parámetros de extracción de características se fijan de la siguiente manera:

El procesamiento de cada archivo sigue estos pasos:

1. **Carga del audio.** Se carga la forma de onda con `librosa.load`, remuestreando a 22050 Hz y convirtiendo a un solo canal (mono). Los archivos demasiado cortos o prácticamente vacíos (menos de 1024 muestras) se descartan.
2. **Transformada de Fourier de tiempo corto (STFT).** Sobre la forma de onda se aplica la STFT con una ventana de Hann de longitud 2048 y un salto de 512 muestras, obteniendo la representación tiempo–frecuencia compleja de la señal.

Parámetro	Valor
Frecuencia de muestreo (SR)	22 050 Hz
Tamaño de la FFT (N_FFT)	2048
Salto entre ventanas (HOP)	512
Número de bandas de Mel (N_MELS)	128
Longitud temporal máxima (MAX_LEN)	128 marcos

Cuadro 1: Parámetros de extracción del espectrograma de Mel.

3. **Espectro de potencia y proyección a la escala de Mel.** Se calcula el espectro de potencia ($|\text{STFT}|^2$) y se multiplica por la matriz de pesos de Mel (`linear_to_mel_weight_matrix`), reduciendo las 1025 bandas de frecuencia lineal a 128 bandas de Mel perceptualmente espaciadas.
4. **Conversión a decibeles y normalización.** El espectrograma de Mel se convierte a escala logarítmica (decibeles) mediante $10 \log_{10}(\text{mel} + 10^{-6})$. Posteriormente se resta el valor máximo (normalización al pico) y se recorta cualquier valor por debajo de -80 dB, limitando el rango dinámico a 80 dB.
5. **Ajuste de longitud fija.** Para que todas las muestras tengan la misma forma, el espectrograma se trunca o se rellena (*padding*) en el eje temporal hasta alcanzar exactamente `MAX_LEN`= 128 marcos. El relleno utiliza el valor de silencio (-80 dB). El resultado es un tensor de tamaño fijo 128×128 (bandas de Mel $\times$ marcos temporales).

La salida de esta etapa es, para cada archivo, una matriz de $128 \times 128 \times 1$, donde el último eje es el canal requerido por las capas `Conv2D`. La etiqueta se codifica como 1 = real y 0 = falso.

3. División de los datos

A diferencia de una partición aleatoria global, este trabajo respeta las divisiones oficiales que el propio conjunto de datos provee en carpetas separadas: `training`, `validation` y `testing`. La función `collect_all` recorre recursivamente el directorio base e identifica, a partir del nombre de cada subcarpeta, tanto la partición (entrenamiento, validación o prueba) como la etiqueta (real o falso), clasificando los archivos de audio en seis grupos:

- Entrenamiento: real / falso
- Validación: real / falso
- Prueba: real / falso

Para mantener un costo computacional manejable se define un objetivo de `TARGET_TOTAL`= 100 000 archivos. Se calcula una fracción de muestreo

$$\text{fraction} = \min\left(1, 0, \frac{\text{TARGET_TOTAL}}{\text{total disponible}}\right),$$

y de cada uno de los seis grupos se toma esa misma fracción de archivos. El muestreo se realiza barajando las listas con una semilla fija (`Random(42)`), lo que garantiza la reproducibilidad del

experimento y preserva la proporción relativa real/falso de cada partición. Respetar las divisiones originales evita la fuga de información (*data leakage*) entre conjuntos y permite que la evaluación sobre el conjunto de prueba sea representativa de datos nunca vistos durante el entrenamiento.

Finalmente, la función `build` convierte cada lista de rutas en los arreglos `X` (espectrogramas) e `y` (etiquetas), descartando con seguridad cualquier archivo que falle durante la extracción de características.

4. Implementación del modelo

4.1. Canalización de datos y aumento (data augmentation)

Los datos se sirven al modelo mediante la API `tf.data`. El conjunto de entrenamiento se procesa con `make_aug_train_ds`, que aplica un aumento de datos por **deformación temporal** (*time warp*): cada espectrograma se estira o comprime aleatoriamente en el eje del tiempo con un factor entre 0.8 y 1.2, simulando variaciones en la velocidad del habla sin alterar el tono (*pitch*). Tras la deformación, la muestra se recorta al centro o se rellena con el valor de silencio para conservar la forma 128×128 . Es importante destacar que este conjunto **no** se almacena en caché, de modo que en cada época se genera una deformación distinta, aumentando la variabilidad efectiva de los datos. Los conjuntos de validación y prueba se sirven con `make_dataset`, sin aumento y con caché en memoria. Se utiliza un tamaño de lote (`BATCH_SIZE`) de 32.

4.2. Arquitectura CNN + BiLSTM

El modelo se construye como una red secuencial de Keras (`build_model`) con la siguiente estructura:

1. **Normalización por muestra.** La capa personalizada `PerSampleStandardization` estandariza cada espectrograma de entrada restando su media y dividiendo por su desviación estándar (normalización Z), estabilizando la escala de las entradas.
2. **Tres bloques convolucionales.** Cada bloque consta de `Conv2D` (con 32, 64 y 128 filtros respectivamente, núcleo 3×3 , *padding same*), seguido de `BatchNormalization`, activación `ReLU` y `MaxPooling2D` con factor 2. Tras los tres submuestreos, la resolución espacial se reduce de 128×128 a 16×16 .
3. **Puente CNN $\rightarrow$ RNN.** Se permutan los ejes para colocar el tiempo primero (`Permute`) y se reorganiza el tensor (`Reshape`) en una secuencia de $T = 16$ pasos temporales, donde cada paso es un vector de $16 \times 128 = 2048$ características.
4. **Dos capas BiLSTM.** Una primera capa `Bidirectional(LSTM(64))` que devuelve la secuencia completa, seguida de una segunda `Bidirectional(LSTM(32))` que resume la secuencia en un único vector. La bidireccionalidad permite que el modelo incorpore contexto tanto pasado como futuro de la señal.
5. **Clasificador.** Una capa `Dense(64, ReLU)`, seguida de `Dropout(0.3)` para regularización, y una capa final `Dense(1, sigmoid)` que produce la probabilidad de que el audio sea real.

4.3. Compilación y configuración del entrenamiento

El modelo se compila con el optimizador **AdamW** (`learning_rate= 0,001`, `weight_decay= 0,004`). AdamW desacopla la regularización por decaimiento de pesos del paso de actualización del gradiente, lo que suele mejorar la generalización frente a Adam clásico [4]. La función de pérdida es la entropía cruzada binaria con *label smoothing* de 0.05, técnica que suaviza las etiquetas objetivo y reduce el exceso de confianza del modelo.

Para contrarrestar el desbalance entre clases, se calculan pesos de clase inversamente proporcionales a la frecuencia de cada etiqueta en el conjunto de entrenamiento (`class_weight`), de modo que la clase minoritaria contribuya proporcionalmente más a la pérdida.

El entrenamiento se ejecuta hasta 30 épocas con dos *callbacks*:

- **EarlyStopping**: monitorea el AUC de validación (`val_auc`), con paciencia de 5 épocas, restaurando los mejores pesos al detenerse.
- **ReduceLROnPlateau**: reduce la tasa de aprendizaje a la mitad cuando la pérdida de validación se estanca durante 3 épocas, hasta un mínimo de 10^{-5} .

5. Evaluación y métricas

Durante el entrenamiento se monitorean cuatro métricas: *accuracy*, precisión (*precision*), exhaustividad (*recall*) y área bajo la curva ROC (*AUC*). El AUC se emplea como criterio principal para la selección del mejor modelo, ya que es robusto frente al desbalance de clases.

La evaluación final sobre el conjunto de prueba comprende:

1. **Métricas globales de prueba.** Con `model.evaluate` se reportan *accuracy*, *precision*, *recall* y AUC sobre el conjunto de prueba.
2. **Curvas de aprendizaje.** Se grafican la pérdida y la exactitud de entrenamiento y validación por época, lo que permite diagnosticar sobreajuste o subajuste.
3. **Predicciones y umbral.** Se obtienen las probabilidades (`model.predict`) y se binarizan con un umbral inicial de 0.5.
4. **Reporte de clasificación.** Mediante `classification_report` se reportan precisión, exhaustividad y F1 por clase (*fake/real*), junto con la F1 macro, que promedia el desempeño de ambas clases por igual.
5. **Optimización del umbral y EER.** Se barren 1 001 umbrales entre 0 y 1 para encontrar el que maximiza la F1 macro. Además, a partir de la curva ROC se calcula la **Tasa de Igual Error** (*Equal Error Rate*, EER), es decir, el punto donde la tasa de falsos positivos iguala a la de falsos negativos; el EER es una métrica estándar en la literatura de detección de audio falso [2].
6. **Matriz de confusión.** Finalmente se visualiza la matriz de confusión sobre el conjunto de prueba, mostrando el conteo de aciertos y errores para cada clase.

6. Conclusión

El cuaderno implementa una canalización completa de detección de audio falso: desde la conversión de las señales en espectrogramas de Mel normalizados, pasando por una división de datos que respeta las particiones oficiales del conjunto, hasta un modelo híbrido CNN–BiLSTM entrenado con AdamW, *label smoothing*, pesos de clase y aumento de datos por deformación temporal. La evaluación se apoya en un conjunto amplio de métricas (*accuracy*, precisión, exhaustividad, F1 macro, AUC y EER) y en visualizaciones que permiten valorar el desempeño del modelo de forma rigurosa.

Referencias

- [1] S. Chapagain, B. Thapa, S. Man Singh Baidhya, S. B.K. y S. Thapa, “Deep Fake Audio Detection Using a Hybrid CNN-BiLSTM Model with Attention Mechanism,” *International Journal on Engineering Technology*, vol. 2, pp. 204–214, 2025. doi: 10.3126/injet.v2i2.78619.
- [2] T. M. Wani, R. Gulzar e I. Amerini, “ABC-CapsNet: Attention based Cascaded Capsule Network for Audio Deepfake Detection,” *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, Seattle, WA, USA, 2024, pp. 2464–2472. doi: 10.1109/CVPRW63382.2024.00253.
- [3] *Audio Deepfake Detection* (preimpresión), arXiv:2509.07132. Disponible en: <https://arxiv.org/abs/2509.07132>.
- [4] Y. Yassin, “Adam vs AdamW: Understanding Weight Decay and Its Impact on Model Performance,” Medium, 2024. Disponible en: <https://yassin01.medium.com/adam-vs-adamw-understanding-weight-decay-and-its-impact-on-model-performance-b7414f0af8a1>.